

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250285425

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

KHURANA; Tarasha et al.

SYSTEM AND METHOD FOR PREDICTING DIVERSE FUTURE GEOMETRIES WITH DIFFUSION MODELS

Abstract

A method of generating data for machine learning (ML) models includes receiving, from one or more sensors, a sequence of samples that includes time-stamp information, extracting from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples, fine-tuning a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples, and in response to the fine-tuning the pre-trained diffusion model to reach convergence, outputting a final-predicted sample associated with the target sample, wherein the final-predicted sample was not in the sequence of samples.

Inventors: KHURANA; Tarasha (Pittsburgh, PA), RAMANAN; Deva (Pittsburgh, PA), MUMMADI; Chaithanya Kumar (Pittsburgh, PA), CABRITA CONDESSA; Filipe J. (Pittsburgh, PA)

Applicant: Robert Bosch GmbH (Stuttgart, DE); Carnegie Mellon University (Pittsburgh, PA)

Family ID: 96949299

Appl. No.: 18/597257

Filed: March 06, 2024

Publication Classification

Int. Cl.: G06V10/82 (20220101); G06V10/77 (20220101); G06V20/40 (20220101)

U.S. Cl.:

CPC G06V10/82 (20220101); G06V10/7715 (20220101); G06V20/46 (20220101);

Background/Summary

TECHNICAL FIELD

[0001] The present disclosure relates to machine learning networks, including those that are utilized for forecasting data.

BACKGROUND

[0002] Modern techniques for generative modeling have pushed the boundaries of 2D and 3D generation, albeit under a static-world assumption. Much evidence from neuroscience suggests that predictive coding to be a fundamental phenomena for biological processing of visual streams, specifically, biological agents process the future by first predicting what may occur and then updating predictions based on actual observations (similar to classic dynamic models such as Kalman filters). Current architectures for video processing tend not to follow such an information flow, such predictive architectures may prove to be fundamental to video processing.

[0003] One of the challenges in operationalizing such a predictive task is that the future is inherently multimodal; consider an outdoor scene of a busy intersection where cars may continue straight or turn. Encoding such uncertainty has been a notorious challenge, but recent generative modeling techniques such as diffusion networks provide an attractive formalism for generating multiple samples from the multimodal future. As such, our work follows a growing body of work on video-based diffusion models. But crucially, rather than generating video samples unconditionally or conditioned on textual prompts, the system may generate future frames conditioned on past observations. But now a significant practical challenge are the compute demands required for learning from massive-scale video datasets.

[0004] Extracting priors from image diffusion models (e.g. Denoising diffusion models) have emerged as an expressive and powerful class of text-to-image generative models. Because of the massive scale of data used to train image diffusion models like Stable Diffusion, Imagen and DALL-e, numerous follow-up works have investigated and built upon the rich representations learnt by them. Specifically within the field of novel-view synthesis, a few works aimed at extracting geometric, pose priors from Stable Diffusion for object or scene-level novel-view-synthesis. Other sparse-view 3D reconstruction works also draw motivation from the same concept for distilling the information from image diffusion into 3D models. A new paradigm of text/image-to-3D assets emerged, where many works iteratively enforced 3D consistency from the outputs of image diffusion models, whereas others repurposed the image models for directly predicting tri-plane representations. There even has been a dedicated study was conducted for understanding the 3D priors learnt by image diffusion models.

[0005] Similarly for the task of video or motion diffusion, some prior systems have attempted to “inflate” image diffusion models to suit video generation, with normalization tricks, a general phenomenon that has appeared before for designing convolutional video understanding architectures. This also extends to the task of 3D motion generation, be it for humans or object trajectories.

[0006] Instead of using image diffusion models, other past systems may try to resource-hungry architectures for 3D video diffusion, including 3D convolutions, and recurrent neural networks, usually coupled with large-scale training datasets. Apart from these, there has been a meteoric rise in recent developments in dedicated text-to-video diffusion models, ranging from industrial-scale pre-training to multi-modality conditioning and generation networks. Some of these methods are even designed for extremely-long autoregressive video generation. The system and method may instead explore the setting where in addition to a moderately-sized data, only limited training resources are available for building a model that conditions on an input timestamp, instead of text.

[0007] The findings from learning self-supervisable representations with masked autoencoders

have recently been adopted by image and video transformer architectures and diffusion models designed for a variety of tasks. Some systems may train in the fashion of masked autoencoders. [0008] In the self-driving world, an important precursor to motion planning is forecasting what the scene and its agents will look like in the future. This spans the field of point cloud, and recently, occupancy forecasting. Forecasting videos of depth has a direct analogue to works that forecast range images of point clouds from LiDAR sensors. For the task of locomotion in robotics, egocentric-depth is increasingly becoming the sole modality that legged robots rely on. This is largely for the reason that depth acts a low-level actionable cue that helps generalization across a vast set of diverse environments for robot navigation. There may be a need to forecast future geometries for use in autonomous systems.

SUMMARY

[0009] A first embodiment discloses a method of generating data for machine learning (ML) models includes receiving, from one or more sensors, a sequence of samples that includes time-stamp information, extracting from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples, fine-tuning a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples, and in response to the fine-tuning the pre-trained diffusion model to reach convergence, outputting a final-predicted sample associated with the target sample, wherein the final-predicted sample was not in the sequence of samples.

[0010] A second embodiment discloses a system that is configured to include one or more sensors configured to retrieve data indicating a sequence of samples. The system includes a controller in communication with the one or more sensors, the controller configured to receive, from one or more sensors, a sequence of samples that includes time-stamp information, extract from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples, fine-tune a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples, in response to the fine-tuning the pre-trained diffusion model to reach convergence, output a final-predicted sample associated with the target sample, wherein the final-predicted sample was not in the sequence of samples.

[0011] A third embodiment discloses, a method of generating data for machine learning (ML) models includes receiving, from one or more sensors, a sequence of samples that includes time-stamp information, extracting from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples, fine-tuning a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples, and in response to the fine-tuning the pre-trained diffusion model reaching a convergence, outputting a final-predicted sample associated with the target sample.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0012] FIG. 1 shows a system for training a neural network, according to an embodiment.

[0013] FIG. 2 shows a computer-implemented method for training and utilizing a neural network, according to an embodiment.

[0014] FIG. 3 illustrates a high-level overview of video prediction architecture.

[0015] FIG. 4 illustrates an embodiment of short horizon forecasting.

[0016] FIG. 5 depicts a schematic diagram of an interaction between a computer-controlled machine and a control system, according to an embodiment.

[0017] FIG. 6 depicts a schematic diagram of the control system of FIG. 5 configured to control a vehicle, which may be a partially autonomous vehicle, a fully autonomous vehicle, a partially autonomous robot, or a fully autonomous robot, according to an embodiment.

[0018] FIG. 7 depicts a schematic diagram of the control system of FIG. 5 configured to control a manufacturing machine, such as a punch cutter, a cutter or a gun drill, of a manufacturing system, such as part of a production line.

[0019] FIG. 8 depicts a schematic diagram of the control system of FIG. 5 configured to control a power tool, such as a power drill or driver, that has an at least partially autonomous mode.

[0020] FIG. 9 depicts a schematic diagram of the control system of FIG. 5 configured to control an automated personal assistant.

[0021] FIG. 10 depicts a schematic diagram of the control system of FIG. 5 configured to control a monitoring system, such as a control access system or a surveillance system.

[0022] FIG. 11 depicts a schematic diagram of the control system of FIG. 5 configured to control an imaging system, for example an MRI apparatus, x-ray imaging apparatus or ultrasonic apparatus.

DETAILED DESCRIPTION

[0023] Embodiments of the present disclosure are described herein. It is to be understood, however, that the disclosed embodiments are merely examples and other embodiments can take various and alternative forms. The figures are not necessarily to scale; some features could be exaggerated or minimized to show details of particular components. Therefore, specific structural and functional details disclosed herein are not to be interpreted as limiting, but merely as a representative bases for teaching one skilled in the art to variously employ the embodiments. As those of ordinary skill in the art will understand, various features illustrated and described with reference to any one of the figures can be combined with features illustrated in one or more other figures to produce embodiments that are not explicitly illustrated or described. The combinations of features illustrated provide representative embodiments for typical application. Various combinations and modifications of the features consistent with the teachings of this disclosure, however, could be desired for particular applications or implementations.

[0024] “A”, “an”, and “the” as used herein refers to both singular and plural referents unless the context clearly dictates otherwise. By way of example, “a processor” programmed to perform various functions refers to one processor programmed to perform each and every function, or more than one processor collectively programmed to perform each of the various functions.

[0025] One of the challenges in operationalizing such a predictive task is that the future is multimodal. For example, consider an outdoor scene of a busy intersection where cars may continue straight or turn. Encoding such uncertainty has been a notorious challenge, but recent generative modeling techniques such as diffusion networks provide an attractive formalism for generating multiple samples from the multimodal future. As such, an embodiment of the system and method below utilizes video-based diffusion models. But rather than generating video samples unconditionally or conditioned on textual prompts, the system may generate future frames conditioned on past observations. One of the significant practical challenge are the compute demands required for learning from massive-scale video datasets.

[0026] In one embodiment, the system may focus on the task of generating (or forecasting) the future from past observations. The given approach relies on at least two key insights. First, the system and method may take the view that accurate video prediction may require understanding of the underlying 3D scene, and so the system and method may base such an approach on networks that perform new-view prediction with conditional diffusion models, such as the Zero-1-to-3 model. While such models are trained to condition on new pose estimates (such as “render this

object from 45 degrees to the left”), the proposed system and method may instead condition on a time stamp. This has numerous advantages. For example, instead of training on multi-view data with ground-truth camera poses (which can be difficult to obtain), the system can train on video frames with timestamps that may be easier to obtain. Moreover, by training on videos with differing framerates, the timestamp-conditioned model can support a variety of video prediction tasks including short horizon forecasting, autoregressive long-horizon forecasting, and even frame interpolation (by conditioning on fractional time-steps).

[0027] Another key insight may be embodied by applications such as robotics and/or self-driving vehicles. Oftentimes, a system may not be not concerned with the photometric properties of the future (e.g., “what will be the color of this car?”) but rather geometric properties (e.g., “where will this car be?”). As such, the system and the method show that it is far more effective to predict (pseudo) depth maps, which can readily be obtained at-scale for videos by running recent monocular depth estimators.

[0028] In summary, the system and method may present a video prediction diffusion network that can be efficiently fine-tuned from foundational image networks. In one embodiment, the system and method may allow a model to be trained on a modest but diverse set of ~1000 videos (e.g., from the TAO dataset), that encompasses a variety of indoor and outdoor scenes, spanning a large vocabulary of objects. The system and method may use a variety of baselines (including nonlinear regression and linear prediction) to illustrate the effectiveness of different modalities, point out that simple regression of depth from historical frames provides a strong baseline for this problem. To illustrate the effectiveness of multimodal forecasting, the system may utilize multimodal (top-K) metrics developed in the forecasting community. The system and method may utilize generative architectures for 3D processing with 2D diffusion models, such as Zero-1-to 3 model, trained on a massive scale of data and repurpose them for the task of timestep-conditioned video prediction.

[0029] Reference is now made to the embodiments illustrated in the Figures, which can apply these teachings to a machine learning model or neural network. FIG. 1 shows a system **100** for training a neural network, e.g. a deep neural network. The system **100** may comprise an input interface for accessing training data **102** for the neural network. For example, as illustrated in FIG. 1, the input interface may be constituted by a data storage interface **104** which may access the training data **102** from a data storage **106**. For example, the data storage interface **104** may be a memory interface or a persistent storage interface, e.g., a hard disk or an SSD interface, but also a personal, local or wide area network interface such as a Bluetooth, Zigbee or Wi-Fi interface or an ethernet or fiberoptic interface. The data storage **106** may be an internal data storage of the system **100**, such as a hard drive or SSD, but also an external data storage, e.g., a network-accessible data storage.

[0030] In some embodiments, the data storage **106** may further comprise a data representation **108** of an untrained version of the neural network which may be accessed by the system **100** from the data storage **106**. It will be appreciated, however, that the training data **102** and the data representation **108** of the untrained neural network may also each be accessed from a different data storage, e.g., via a different subsystem of the data storage interface **104**. Each subsystem may be of a type as is described above for the data storage interface **104**. In other embodiments, the data representation **108** of the untrained neural network may be internally generated by the system **100** on the basis of design parameters for the neural network, and therefore may not explicitly be stored on the data storage **106**. The system **100** may further comprise a processor subsystem **110** which may be configured to, during operation of the system **100**, provide an iterative function as a substitute for a stack of layers of the neural network to be trained. Here, respective layers of the stack of layers being substituted may have mutually shared weights and may receive as input an output of a previous layer, or for a first layer of the stack of layers, an initial activation, and a part of the input of the stack of layers. The processor subsystem **110** may be further configured to iteratively train the neural network using the training data **102**. Here, an iteration of the training by the processor subsystem **110** may comprise a forward propagation part and a backward propagation

part. The processor subsystem **110** may be configured to perform the forward propagation part by, amongst other operations defining the forward propagation part which may be performed, determining an equilibrium point of the iterative function at which the iterative function converges to a fixed point, wherein determining the equilibrium point comprises using a numerical root-finding algorithm to find a root solution for the iterative function minus its input, and by providing the equilibrium point as a substitute for an output of the stack of layers in the neural network. The system **100** may further comprise an output interface for outputting a data representation **112** of the trained neural network, this data may also be referred to as trained model data **112**. For example, as also illustrated in FIG. 1, the output interface may be constituted by the data storage interface **104**, with said interface being in these embodiments an input/output ('IO') interface, via which the trained model data **112** may be stored in the data storage **106**. For example, the data representation **108** defining the 'untrained' neural network may during or after the training be replaced, at least in part by the data representation **112** of the trained neural network, in that the parameters of the neural network, such as weights, hyperparameters and other types of parameters of neural networks, may be adapted to reflect the training on the training data **102**. This is also illustrated in FIG. 1 by the reference numerals **108**, **112** referring to the same data record on the data storage **106**. In other embodiments, the data representation **112** may be stored separately from the data representation **108** defining the 'untrained' neural network. In some embodiments, the output interface may be separate from the data storage interface **104**, but may in general be of a type as described above for the data storage interface **104**.

[0031] The structure of the system **100** is one example of a system that may be utilized to train the machine-learning model described herein. Additional structure for operating and training the machine-learning models is shown in FIG. 2.

[0032] FIG. 2 depicts a system **200** to implement the machine-learning models described herein. The system **200** can be implemented to predict diverse future geometries with a diffusion model, as described herein. The system **200** may include at least one computing system **202**. The computing system **202** may include at least one processor **204** that is operatively connected to a memory unit **208**. The processor **204** may include one or more integrated circuits that implement the functionality of a central processing unit (CPU) **206**. The CPU **206** may be a commercially available processing unit that implements an instruction set such as one of the x86, ARM, Power, or MIPS instruction set families. During operation, the CPU **206** may execute stored program instructions that are retrieved from the memory unit **208**. The stored program instructions may include software that controls operation of the CPU **206** to perform the operation described herein. In some examples, the processor **204** may be a system on a chip (SoC) that integrates functionality of the CPU **206**, the memory unit **208**, a network interface, and input/output interfaces into a single integrated device. The computing system **202** may implement an operating system for managing various aspects of the operation. While one processor **204**, one CPU **206**, and one memory **208** is shown in FIG. 2, of course more than one of each can be utilized in an overall system.

[0033] The memory unit **208** may include volatile memory and non-volatile memory for storing instructions and data. The non-volatile memory may include solid-state memories, such as NAND flash memory, magnetic and optical storage media, or any other suitable data storage device that retains data when the computing system **202** is deactivated or loses electrical power. The volatile memory may include static and dynamic random-access memory (RAM) that stores program instructions and data. For example, the memory unit **208** may store a machine-learning model **210** or algorithm, a training dataset **212** for the machine-learning model **210**, raw source dataset **216**.

[0034] The computing system **202** may include a network interface device **222** that is configured to provide communication with external systems and devices. For example, the network interface device **222** may include a wired and/or wireless Ethernet interface as defined by Institute of Electrical and Electronics Engineers (IEEE) 802.11 family of standards. The network interface device **222** may include a cellular communication interface for communicating with a cellular

network (e.g., 3G, 4G, 5G). The network interface device **222** may be further configured to provide a communication interface to an external network **224** or cloud.

[0035] The external network **224** may be referred to as the world-wide web or the Internet. The external network **224** may establish a standard communication protocol between computing devices. The external network **224** may allow information and data to be easily exchanged between computing devices and networks. One or more servers **230** may be in communication with the external network **224**.

[0036] The computing system **202** may include an input/output (I/O) interface **220** that may be configured to provide digital and/or analog inputs and outputs. The I/O interface **220** is used to transfer information between internal storage and external input and/or output devices (e.g., HMI devices). The I/O **220** interface can include associated circuitry or BUS networks to transfer information to or between the processor(s) and storage. For example, the I/O interface **220** can include digital I/O logic lines which can be read or set by the processor(s), handshake lines to supervise data transfer via the I/O lines; timing and counting facilities, and other structure known to provide such functions. Examples of input devices include a keyboard, mouse, sensors, etc. Examples of output devices include monitors, printers, speakers, etc. The I/O interface **220** may include additional serial interfaces for communicating with external devices (e.g., Universal Serial Bus (USB) interface).

[0037] The computing system **202** may include a human-machine interface (HMI) device **218** that may include any device that enables the system **200** to receive control input. Examples of input devices may include human interface inputs such as keyboards, mice, touchscreens, voice input devices, and other similar devices. The computing system **202** may include a display device **232**. The computing system **202** may include hardware and software for outputting graphics and text information to the display device **232**. The display device **232** may include an electronic display screen, projector, printer or other suitable device for displaying information to a user or operator. The computing system **202** may be further configured to allow interaction with remote HMI and remote display devices via the network interface device **222**.

[0038] The system **200** may be implemented using one or multiple computing systems. While the example depicts a single computing system **202** that implements all of the described features, it is intended that various features and functions may be separated and implemented by multiple computing units in communication with one another. The particular system architecture selected may depend on a variety of factors.

[0039] The system **200** may implement a machine-learning algorithm **210** that is configured to analyze the raw source dataset **216**. The raw source dataset **216** may include raw or unprocessed sensor data that may be representative of an input dataset for a machine-learning system. The raw source dataset **216** may include video, video segments, images, text-based information, audio or human speech, time series data (e.g., a pressure sensor signal over time or time-stamped video data), and raw or partially processed sensor data (e.g., radar map of objects). Several different examples of inputs are shown and described with reference to FIGS. 5-11. In some examples, the machine-learning algorithm **210** may be a neural network algorithm (e.g., deep neural network) that is designed to perform a predetermined function. For example, the neural network algorithm may be configured in automotive applications to identify street signs or pedestrians in images. The machine-learning algorithm(s) **210** may include algorithms configured to operate the models described herein.

[0040] The computer system **200** may store a training dataset **212** for the machine-learning algorithm **210**. The training dataset **212** may represent a set of previously constructed data for training the machine-learning algorithm **210**. The training dataset **212** may be used by the machine-learning algorithm **210** to learn weighting factors associated with a neural network algorithm. The training dataset **212** may include a set of source data that has corresponding outcomes or results that the machine-learning algorithm **210** tries to duplicate via the learning process. In this example,

the training dataset **212** may include input images that include an object (e.g., a street sign). The input images may include various scenarios in which the objects are identified.

[0041] The machine-learning algorithm **210** may be operated in a learning mode using the training dataset **212** as input. The machine-learning algorithm **210** may be executed over a number of iterations using the data from the training dataset **212**. With each iteration, the machine-learning algorithm **210** may update internal weighting factors based on the achieved results. For example, the machine-learning algorithm **210** can compare output results (e.g., a reconstructed or supplemented image, in the case where image data is the input) with those included in the training dataset **212**. Since the training dataset **212** includes the expected results, the machine-learning algorithm **210** can determine when performance is acceptable. After the machine-learning algorithm **210** achieves a predetermined performance level (e.g., 100% agreement with the outcomes associated with the training dataset **212**), or convergence, the machine-learning algorithm **210** may be executed using data that is not in the training dataset **212**. It should be understood that in this disclosure, “convergence” can mean a set (e.g., predetermined) number of iterations have occurred, or that the residual is sufficiently small (e.g., the change in the approximate probability over iterations is changing by less than a threshold), or other convergence conditions. The trained machine-learning algorithm **210** may be applied to new datasets to generate annotated data.

[0042] The machine-learning algorithm **210** may be configured to identify a particular feature in the raw source data **216**. The raw source data **216** may include a plurality of instances or input dataset for which supplementation results are desired. For example, the machine-learning algorithm **210** may be configured to identify the presence of a road sign in video images and annotate the occurrences. The machine-learning algorithm **210** may be programmed to process the raw source data **216** to identify the presence of the particular features. The machine-learning algorithm **210** may be configured to identify a feature in the raw source data **216** as a predetermined feature (e.g., road sign). The raw source data **216** may be derived from a variety of sources. For example, the raw source data **216** may be actual input data collected by a machine-learning system. The raw source data **216** may be machine generated for testing the system. As an example, the raw source data **216** may include raw video images from a camera.

[0043] In an example, the raw source data **216** may include image data representing an image. Applying the machine-learning algorithms described herein, the system and method may predict diverse future geometries with a diffusion model.

[0044] FIG. 3 illustrates a high-level overview of video prediction architecture. The system may be a multi-frame conditioning architecture **300** that also listens to the time stamps of context views and generates a generated output per the time stamp of a target view. The system and method may utilize generative architectures for 3D processing with 2D diffusion models, such as Zero-1-to-3 model, trained on a massive scale of data and repurpose them for the task of timestep-conditioned video prediction.

[0045] The system may include a set of sample inputs **301**. The sample inputs may include a time stamp reflective of various images or frames. One of the frames **301** may be pure noise (e.g., frame at -0.46 s). Given a set of context frames $c \in \mathbb{R}^{1 \times H \times W \times C}$ from a video of a dynamic scene, one goal may be to generate a frame $x \in \mathbb{R}^{1 \times H \times W \times C}$ for the same scene but from a different point in time. Let all timesteps in consideration be $t = [t_{\text{sub.c.sup.1}}, \dots, t_{\text{sub.c.sup.T-1}}, t_{\text{sub.x}}] \in \mathbb{R}^T$. Then, the system may want to learn a function g that generates an estimate of the unobserved x given c and t .

[00001] $\hat{x} = g(x, t)$ (Equation1)

[0046] Since, $\{\text{circumflex over } (x)\}$ is unobserved, it inherently follows a multimodal distribution, making its prediction underconstrained. To this end, the system and method may exploit pretrained large diffusion models like Stable Diffusion that can model and sample from such multimodal distributions of natural images. The sample inputs **301** may be fed into a diffusion model. It is not

unreasonable to use single-frame 2D diffusion models for the task of video prediction, as their large-scale pre-training likely covers the space of temporal events and the different stages of their unfolding. Time-conditioned video prediction demands may require two new capabilities: first, the ability to generate a new frame that is consistent with the historical context frames c , and second, the ability to listen to the continuous valued timesteps, t .

[0047] Given the above, the system may formalize the task of video prediction in the context of diffusion models as set forth below. Given a dataset of videos with a known frame per second (FPS), the system may extract snippets of length T and construct a training sample as $\{x, c, t\}$. Using this training data, the system may start from the natural scene-level data distribution learnt by Stable Diffusion Image Variations and fine-tune it for controlling both the conditioning with the context frames, and timestep scalars. Architecturally similar to Zero-1-to-3, the system and method may use a denoising UNet $\epsilon_{\text{sub}, \theta}$, that looks at 64×64 images. For any timestep $i \in [0, 1000]$, the system may train $\epsilon_{\text{sub}, \theta}$, with the well-adopted noise prediction objective for diffusion training,

$$[00002] \min_{\epsilon_{\text{sub}, \theta}} \mathbb{E}_{z \sim x, i, \epsilon \sim N(0, 1)} \|\epsilon - \epsilon_{\text{sub}, \theta}(z_i, i, f(x, c, t))\|_2^2 \quad (\text{Equation 2})$$

Where $f(x, c, t)$ is the conditioning embedding obtained by modifications to Zero-1-to-3 detailed in the following subsections. At inference, the system may start from pure gaussian noise, and iteratively denoise it, steering the denoised image in the direction of the conditioning embedding. Eventually, a target output **305** may be output based on the model reaching convergence (e.g., minimizing loss or meeting a specific threshold for loss).

[0048] For conditioning on context views, the system may follow Zero-1-to-3 or a similar approach. In its two-stream image conditioning protocol but modify it to suit our multi-frame setting. For conditioning on low-level features of the input context frames (such as depth, texture, and motion patterns of scene actors), the system may concatenate the $T-1$ frames with the noisy input image to the UNet. For conditioning on higher-level features of the input context frames (such as the scene elements, contextual background, and observed camera trajectory), the system may pass the context frames through the CLIP image encoder to get their image embeddings. The system may additionally construct a $T_{\text{sup}, \text{th}}$ “residual” CLIP embedding for the target frame, by learning the weights on $T-1$ embeddings and taking their weighted average. Intuitively, this “guides” the target image **305** with a residual embedding that can be hooked onto, in order to generate the prediction.

[0049] The system may be conditioned on timestep scalars. In addition to building a conditioning mechanism for the context views, the system may also need to let the denoising UNet know, which timesteps the context frames belonged to, and which timestep to be are probing for. To accomplish this, the system may positionally encode the timestep scalar with sinusoidal embeddings,

$$[00003] \quad (t) = (\sin 2^0 t, \cos 2^0 t, \sin 2^{L-1} t, \cos 2^{L-1} t) \quad (3)$$

[0050] This ensures that even if every timestamp value is not seen during training, any high-frequency variation of it can be approximated in the frequency domain at inference. The system may concatenate this time embedding with the CLIP embeddings, and cross-attend them at every residual block in the UNet architecture. Even though at inference this method addresses forecasting, the system may be trained for a ‘random timestep prediction’ objective, instead of the task of forecasting itself.

[0051] The system and method may train an architecture with classifier free guidance, implying that the system may randomly remove the conditioning so as to allow the model to generate unconditional frames (which in turn can be used as a guidance signal during inference). During training, the system may predict the noise added to the input sample, and set the probability of dropping the conditioning for classifier free guidance to 10%. During inference, the system and method may utilize a guidance of 2.0 for all experiments, with DDPM sampling for iterative denoising steps (e.g. forty steps). Unlike, Zero-1-to-3 that uses other samplers and a guidance in the range of hundreds, such a setting may be optimal in the present embodiment. The system and

method may not perform diffusion in the latent space, but train and evaluate on images of size 64×64 using just the Stable Diffusion Image Variations UNet. To circumvent the use of VAE, the system may learn two new convolutional layers at the start and end of the UNet that help the input image to adjust to the weights of the UNet learnt for diffusion in the latent space, similar to other methods that do not depend on the VAE. The system may learn all new layers 10× faster than the other layers, for training from scratch. The system and method may train the network with a batch size of 12 for 10 k iterations, using AdamW with $\beta_{\text{sub.1}}=0.95$, $\beta_{\text{sub.2}}=0.999$, $\epsilon=1\text{e.sup.}-8$ and weight decay of $1\text{e.sup.}-6$.

[0052] FIG. 4 illustrates an embodiment of short horizon forecasting. FIG. 4 discloses some examples of input-output-ground truth triples. Given the previous 3 frames that are input as grayscale, the system may show 3 different samples of the future from the diffusion network, and the corresponding ground truth in the video logs. The sample from the model is highlighted with boundary boxes as shown in FIG. 4. In addition to producing diverse futures, the system and method may listen to low-level details in the historical context frames (e.g., scene structure, actors, and overall camera motion) and provide multiple realistic predictions of the multimodal future, whether the scene is dynamic or stationary (as in the two examples in the last row).

[0053] There are several examples of what applications to the system and method may be utilized for. For example, in an approach to predictive video modelling with a controllable time step conditioning can be used for long-horizon forecasting, temporal super-resolution (e.g., interpolation with fractional time steps), and back-casting given the future. With long-horizon forecasting, the choice of input time steps may allow one to generate autoregressive long-horizon videos of varying framerates given a context of frames. With temporal super-resolution, the system may allow, based on the nature of the time step conditioning, a query to the diffusion network on fractional time stamps in order to interpolate between a set of frames. This may result in an application of temporal super-resolution that given a video at a lower framerate, one can generate a same video at a higher framerate. With respect to back-casting, the system may, given a current and future frame, be able to “look back” in the past and generate long-horizon back-casting videos.

[0054] The machine-learning models described herein can be used in many different applications, and not just in the context of road sign image processing. Additional applications where anomaly detection or classification may be used are shown in FIGS. 6-11. Structure used for training and using the machine-learning models for these applications (and other applications) are exemplified in FIG. 5. FIG. 5 depicts a schematic diagram of an interaction between a computer-controlled machine 500 and a control system 502. Computer-controlled machine 500 includes actuator 504 and sensor 506. Actuator 504 may include one or more actuators and sensor 506 may include one or more sensors. Sensor 506 is configured to sense a condition of computer-controlled machine 500. Sensor 506 may be configured to encode the sensed condition into sensor signals 508 and to transmit sensor signals 508 to control system 502. Non-limiting examples of sensor 506 include video, radar, LiDAR, ultrasonic and motion sensors. In one embodiment, sensor 506 is an optical sensor configured to sense optical images of an environment proximate to computer-controlled machine 500.

[0055] Control system 502 is configured to receive sensor signals 508 from computer-controlled machine 500. As set forth below, control system 502 may be further configured to compute actuator control commands 510 depending on the sensor signals and to transmit actuator control commands 510 to actuator 504 of computer-controlled machine 500.

[0056] As shown in FIG. 5, control system 502 includes receiving unit 512. Receiving unit 512 may be configured to receive sensor signals 508 from sensor 506 and to transform sensor signals 508 into input signals x. In an alternative embodiment, sensor signals 508 are received directly as input signals x without receiving unit 512. Each input signal x may be a portion of each sensor signal 508. Receiving unit 512 may be configured to process each sensor signal 508 to product each input signal x. Input signal x may include data corresponding to an image recorded by sensor

506.

[0057] Control system **502** includes a classifier **514**. Classifier **514** may be configured to classify input signals x into one or more labels using a machine learning (ML) algorithm, such as a neural network described above. Classifier **514** is configured to be parametrized by parameters, such as those described above (e.g., parameter θ). Parameters θ may be stored in and provided by non-volatile storage **516**. Classifier **514** is configured to determine output signals y from input signals x . Each output signal y includes information that assigns one or more labels to each input signal x . Classifier **514** may transmit output signals y to conversion unit **518**. Conversion unit **518** is configured to convert output signals y into actuator control commands **510**. Control system **502** is configured to transmit actuator control commands **510** to actuator **504**, which is configured to actuate computer-controlled machine **500** in response to actuator control commands **510**. In another embodiment, actuator **504** is configured to actuate computer-controlled machine **500** based directly on output signals y .

[0058] Upon receipt of actuator control commands **510** by actuator **504**, actuator **504** is configured to execute an action corresponding to the related actuator control command **510**. Actuator **504** may include a control logic configured to transform actuator control commands **510** into a second actuator control command, which is utilized to control actuator **504**. In one or more embodiments, actuator control commands **510** may be utilized to control a display instead of or in addition to an actuator.

[0059] In another embodiment, control system **502** includes sensor **506** instead of or in addition to computer-controlled machine **500** including sensor **506**. Control system **502** may also include actuator **504** instead of or in addition to computer-controlled machine **500** including actuator **504**.

[0060] As shown in FIG. 5, control system **502** also includes processor **520** and memory **522**. Processor **520** may include one or more processors. Memory **522** may include one or more memory devices. The classifier **514** (e.g., machine-learning algorithms, such as those described above with regard to pre-trained classifier **306**) of one or more embodiments may be implemented by control system **502**, which includes non-volatile storage **516**, processor **520** and memory **522**.

[0061] Non-volatile storage **516** may include one or more persistent data storage devices such as a hard drive, optical drive, tape drive, non-volatile solid-state device, cloud storage or any other device capable of persistently storing information. Processor **520** may include one or more devices selected from high-performance computing (HPC) systems including high-performance cores, microprocessors, micro-controllers, digital signal processors, microcomputers, central processing units, field programmable gate arrays, programmable logic devices, state machines, logic circuits, analog circuits, digital circuits, or any other devices that manipulate signals (analog or digital) based on computer-executable instructions residing in memory **522**. Memory **522** may include a single memory device or a number of memory devices including, but not limited to, random access memory (RAM), volatile memory, non-volatile memory, static random access memory (SRAM), dynamic random access memory (DRAM), flash memory, cache memory, or any other device capable of storing information.

[0062] Processor **520** may be configured to read into memory **522** and execute computer-executable instructions residing in non-volatile storage **516** and embodying one or more ML algorithms and/or methodologies of one or more embodiments. Non-volatile storage **516** may include one or more operating systems and applications. Non-volatile storage **516** may store compiled and/or interpreted from computer programs created using a variety of programming languages and/or technologies, including, without limitation, and either alone or in combination, Java, C, C++, C#, Objective C, Fortran, Pascal, Java Script, Python, Perl, and PL/SQL.

[0063] Upon execution by processor **520**, the computer-executable instructions of non-volatile storage **516** may cause control system **502** to implement one or more of the ML algorithms and/or methodologies as disclosed herein. Non-volatile storage **516** may also include ML data (including data parameters) supporting the functions, features, and processes of the one or more embodiments

described herein.

[0064] The program code embodying the algorithms and/or methodologies described herein is capable of being individually or collectively distributed as a program product in a variety of different forms. The program code may be distributed using a computer readable storage medium having computer readable program instructions thereon for causing a processor to carry out aspects of one or more embodiments. Computer readable storage media, which is inherently non-transitory, may include volatile and non-volatile, and removable and non-removable tangible media implemented in any method or technology for storage of information, such as computer-readable instructions, data structures, program modules, or other data. Computer readable storage media may further include RAM, ROM, erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), flash memory or other solid state memory technology, portable compact disc read-only memory (CD-ROM), or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium that can be used to store the desired information and which can be read by a computer. Computer readable program instructions may be downloaded to a computer, another type of programmable data processing apparatus, or another device from a computer readable storage medium or to an external computer or external storage device via a network.

[0065] Computer readable program instructions stored in a computer readable medium may be used to direct a computer, other types of programmable data processing apparatus, or other devices to function in a particular manner, such that the instructions stored in the computer readable medium produce an article of manufacture including instructions that implement the functions, acts, and/or operations specified in the flowcharts or diagrams. In certain alternative embodiments, the functions, acts, and/or operations specified in the flowcharts and diagrams may be re-ordered, processed serially, and/or processed concurrently consistent with one or more embodiments. Moreover, any of the flowcharts and/or diagrams may include more or fewer nodes or blocks than those illustrated consistent with one or more embodiments.

[0066] The processes, methods, or algorithms can be embodied in whole or in part using suitable hardware components, such as Application Specific Integrated Circuits (ASICs), Field-Programmable Gate Arrays (FPGAs), state machines, controllers or other hardware components or devices, or a combination of hardware, software and firmware components.

[0067] FIG. 6 depicts a schematic diagram of control system 502 configured to control vehicle 600, which may be an at least partially autonomous vehicle or an at least partially autonomous robot. Vehicle 600 includes actuator 504 and sensor 506. Sensor 506 may include one or more video sensors, cameras, radar sensors, ultrasonic sensors, LiDAR sensors, and/or position sensors (e.g. GPS). One or more of the one or more specific sensors may be integrated into vehicle 600. In the context of sign-recognition and processing as described herein, the sensor 506 is a camera mounted to or integrated into the vehicle 600. Alternatively or in addition to one or more specific sensors identified above, sensor 506 may include a software module configured to, upon execution, determine a state of actuator 504. One non-limiting example of a software module includes a weather information software module configured to determine a present or future state of the weather proximate vehicle 600 or other location.

[0068] Classifier 514 of control system 502 of vehicle 600 may be configured to detect objects in the vicinity of vehicle 600 dependent on input signals x. In such an embodiment, output signal y may include information characterizing the vicinity of objects to vehicle 600. Actuator control command 510 may be determined in accordance with this information. The actuator control command 510 may be used to avoid collisions with the detected objects.

[0069] In embodiments where vehicle 600 is an at least partially autonomous vehicle, actuator 504 may be embodied in a brake, a propulsion system, an engine, a drivetrain, or a steering of vehicle 600. Actuator control commands 510 may be determined such that actuator 504 is controlled such that vehicle 600 avoids collisions with detected objects. Detected objects may also be classified

according to what classifier **514** deems them most likely to be, such as pedestrians or trees. The actuator control commands **510** may be determined depending on the classification. In a scenario where an adversarial attack may occur, the system described above may be further trained to better detect objects or identify a change in lighting conditions or an angle for a sensor or camera on vehicle **600**.

[0070] In other embodiments where vehicle **600** is an at least partially autonomous robot, vehicle **600** may be a mobile robot that is configured to carry out one or more functions, such as flying, swimming, diving and stepping. The mobile robot may be an at least partially autonomous lawn mower or an at least partially autonomous cleaning robot. In such embodiments, the actuator control command **510** may be determined such that a propulsion unit, steering unit and/or brake unit of the mobile robot may be controlled such that the mobile robot may avoid collisions with identified objects.

[0071] In another embodiment, vehicle **600** is an at least partially autonomous robot in the form of a gardening robot. In such embodiment, vehicle **600** may use an optical sensor as sensor **506** to determine a state of plants in an environment proximate vehicle **600**. Actuator **504** may be a nozzle configured to spray chemicals. Depending on an identified species and/or an identified state of the plants, actuator control command **510** may be determined to cause actuator **504** to spray the plants with a suitable quantity of suitable chemicals.

[0072] Vehicle **600** may be an at least partially autonomous robot in the form of a domestic appliance. Non-limiting examples of domestic appliances include a washing machine, a stove, an oven, a microwave, or a dishwasher. In such a vehicle **600**, sensor **506** may be an optical sensor configured to detect a state of an object which is to undergo processing by the household appliance. For example, in the case of the domestic appliance being a washing machine, sensor **506** may detect a state of the laundry inside the washing machine. Actuator control command **510** may be determined based on the detected state of the laundry.

[0073] FIG. 7 depicts a schematic diagram of control system **502** configured to control system **700** (e.g., manufacturing machine), such as a punch cutter, a cutter or a gun drill, of manufacturing system **702**, such as part of a production line. Control system **502** may be configured to control actuator **504**, which is configured to control system **700** (e.g., manufacturing machine).

[0074] Sensor **506** of system **700** (e.g., manufacturing machine) may be an optical sensor configured to capture one or more properties of manufactured product **704**. Classifier **514** may be configured to determine a state of manufactured product **704** from one or more of the captured properties. Actuator **504** may be configured to control system **700** (e.g., manufacturing machine) depending on the determined state of manufactured product **704** for a subsequent manufacturing step of manufactured product **704**. The actuator **504** may be configured to control functions of system **700** (e.g., manufacturing machine) on subsequent manufactured product **106** of system **700** (e.g., manufacturing machine) depending on the determined state of manufactured product **704**.

[0075] FIG. 8 depicts a schematic diagram of control system **502** configured to control power tool **800**, such as a power drill or driver, that has an at least partially autonomous mode. Control system **502** may be configured to control actuator **504**, which is configured to control power tool **800**.

[0076] Sensor **506** of power tool **800** may be an optical sensor configured to capture one or more properties of work surface **802** and/or fastener **804** being driven into work surface **802**. Classifier **514** may be configured to determine a state of work surface **802** and/or fastener **804** relative to work surface **802** from one or more of the captured properties. The state may be fastener **804** being flush with work surface **802**. The state may alternatively be hardness of work surface **802**. Actuator **504** may be configured to control power tool **800** such that the driving function of power tool **800** is adjusted depending on the determined state of fastener **804** relative to work surface **802** or one or more captured properties of work surface **802**. For example, actuator **504** may discontinue the driving function if the state of fastener **804** is flush relative to work surface **802**. As another non-limiting example, actuator **504** may apply additional or less torque depending on the hardness of

work surface **802**.

[0077] FIG. **9** depicts a schematic diagram of control system **502** configured to control automated personal assistant **900**. Control system **502** may be configured to control actuator **504**, which is configured to control automated personal assistant **900**. Automated personal assistant **900** may be configured to control a domestic appliance, such as a washing machine, a stove, an oven, a microwave or a dishwasher.

[0078] Sensor **506** may be an optical sensor and/or an audio sensor. The optical sensor may be configured to receive video images of gestures **904** of user **902**. The audio sensor may be configured to receive a voice command of user **902**.

[0079] Control system **502** of automated personal assistant **900** may be configured to determine actuator control commands **510** configured to control system **502**. Control system **502** may be configured to determine actuator control commands **510** in accordance with sensor signals **508** of sensor **506**. Automated personal assistant **900** is configured to transmit sensor signals **508** to control system **502**. Classifier **514** of control system **502** may be configured to execute a gesture recognition algorithm to identify gesture **904** made by user **902**, to determine actuator control commands **510**, and to transmit the actuator control commands **510** to actuator **504**. Classifier **514** may be configured to retrieve information from non-volatile storage in response to gesture **904** and to output the retrieved information in a form suitable for reception by user **902**.

[0080] FIG. **10** depicts a schematic diagram of control system **502** configured to control monitoring system **1000**. Monitoring system **1000** may be configured to physically control access through door **1002**. Sensor **506** may be configured to detect a scene that is relevant in deciding whether access is granted. Sensor **506** may be an optical sensor configured to generate and transmit image and/or video data. Such data may be used by control system **502** to detect a person's face.

[0081] Classifier **514** of control system **502** of monitoring system **1000** may be configured to interpret the image and/or video data by matching identities of known people stored in non-volatile storage **516**, thereby determining an identity of a person. Classifier **514** may be configured to generate and an actuator control command **510** in response to the interpretation of the image and/or video data. Control system **502** is configured to transmit the actuator control command **510** to actuator **504**. In this embodiment, actuator **504** may be configured to lock or unlock door **1002** in response to the actuator control command **510**. In other embodiments, a non-physical, logical access control is also possible.

[0082] Monitoring system **1000** may also be a surveillance system. In such an embodiment, sensor **506** may be an optical sensor configured to detect a scene that is under surveillance and control system **502** is configured to control display **1004**. Classifier **514** is configured to determine a classification of a scene, e.g. whether the scene detected by sensor **506** is suspicious. Control system **502** is configured to transmit an actuator control command **510** to display **1004** in response to the classification. Display **1004** may be configured to adjust the displayed content in response to the actuator control command **510**. For instance, display **1004** may highlight an object that is deemed suspicious by classifier **514**. Utilizing an embodiment of the system disclosed, the surveillance system may predict objects at certain times in the future showing up.

[0083] FIG. **11** depicts a schematic diagram of control system **502** configured to control imaging system **1100**, for example an MRI apparatus, x-ray imaging apparatus or ultrasonic apparatus. Sensor **506** may, for example, be an imaging sensor. Classifier **514** may be configured to determine a classification of all or part of the sensed image. Classifier **514** may be configured to determine or select an actuator control command **510** in response to the classification obtained by the trained neural network. For example, classifier **514** may interpret a region of a sensed image to predict future images. In this case, actuator control command **510** may be determined or selected to cause display **1102** to display the imaging and predict future images, or back-cast images.

[0084] While exemplary embodiments are described above, it is not intended that these embodiments describe all possible forms encompassed by the claims. The words used in the

specification are words of description rather than limitation, and it is understood that various changes can be made without departing from the spirit and scope of the disclosure. As previously described, the features of various embodiments can be combined to form further embodiments of the invention that may not be explicitly described or illustrated. While various embodiments could have been described as providing advantages or being preferred over other embodiments or prior art implementations with respect to one or more desired characteristics, those of ordinary skill in the art recognize that one or more features or characteristics can be compromised to achieve desired overall system attributes, which depend on the specific application and implementation. These attributes can include, but are not limited to cost, strength, durability, life cycle cost, marketability, appearance, packaging, size, serviceability, weight, manufacturability, ease of assembly, etc. As such, to the extent any embodiments are described as less desirable than other embodiments or prior art implementations with respect to one or more characteristics, these embodiments are not outside the scope of the disclosure and can be desirable for particular applications.

Claims

1. A method of generating data for machine learning (ML) models, the method comprising: receiving, from one or more sensors, a sequence of samples that includes time-stamp information; extracting from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples; fine-tuning a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples; in response to the fine-tuning the pre-trained diffusion model to reach convergence, outputting a final-predicted sample associated with the target sample, wherein the final-predicted sample was not in the sequence of samples.
2. The method of claim 1, wherein the context associated with the sequence of samples is one less than all samples of the sequence of samples.
3. The method of claim 1, wherein fine-tuning the pre-trained diffusion model includes iteratively denoising Gaussian noise associated with the context.
4. The method of claim 1, wherein the sequence of samples includes video data with one or more frames of video.
5. The method of claim 1, wherein the final predicted sample is in a future time outside of the sequence of samples.
6. The method of claim 1, wherein the final-predicted sample that is a super-resolution sample generating a higher framerate than the sequence of samples.
7. The method of claim 1, wherein the pre-trained diffusion model is a Stable Diffusion model.
8. The method of claim 1, wherein the final-predicated sample is a long-horizon backcasting video.
9. A system, comprising: one or more sensors configured to retrieve data indicating a sequence of samples; and a controller in communication with the one or more sensors, the controller configured to: receive, from one or more sensors, a sequence of samples that includes time-stamp information; extract from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples; fine-tune a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples; in response to the fine-tuning the pre-trained diffusion model to reach convergence, output a final-predicted sample associated with the target sample, wherein the final-predicted sample was not in the sequence of samples.

- 10.** The system of claim 9, wherein the one or more sensors includes a camera configured to generate video data include a plurality of frames.
 - 11.** The system of claim 9, wherein the pre-trained diffusion model is a Stable Diffusion model.
 - 12.** The system of claim 9, wherein fine-tuning the pre-trained diffusion model includes iteratively denoising utilizing a denoising model.
 - 13.** The system of claim 9, wherein the final-predicted sample is generated to duplicate the target sample.
 - 14.** The system of claim 9, wherein the final-predicted sample that is in a future time outside of the sequence of samples.
 - 15.** The system of claim 9, wherein the final-predicted sample is a super-resolution sample generating a higher framerate than the sequence of samples.
 - 16.** The system of claim 9, wherein the target sample is pure Gaussian noise.
 - 17.** A method of generating data for machine learning (ML) models, the method comprising: receiving, from one or more sensors, a sequence of samples that includes time-stamp information; extracting from the sequence of samples a snippet of a pre-defined length to generate a training dataset that includes a target sample derived from the sequence of samples; fine-tuning a pre-trained diffusion model to condition based on a context sample associated with the sequence of samples and corresponding time-stamp information, wherein the context sample associated with the sequence of samples is less than all samples of the sequence of samples; and in response to the fine-tuning the pre-trained diffusion model reaching a convergence, outputting a final-predicted sample associated with the target sample.
 - 18.** The method of claim 17, wherein the final-predicted sample was not in the sequence of samples.
 - 19.** The method of claim 17, wherein the final-predicted sample is generated to duplicate the target sample.
 - 20.** The method of claim 17, wherein the target sample is pure Gaussian noise that includes an associated timestamp.
-